

Alpaca AI Trading Agents Hackathon - Project Documentation

Author: Oresti Janko | **Event:** Alpaca AI Trading Agents Hackathon (lablab.ai) | **Window:** Aug 28 - Sep 4, 2026

This document is the running record of what was built and why. It's the source material for the one-page write-up, slide deck, and video script - not a polished submission itself.

1. Strategy Overview

Approach: Statistical pairs trading, expressed through options rather than raw equity positions.

Core thesis: Find pairs of S&P 500 stocks whose prices have historically moved together (correlated and cointegrated). When the price relationship between the two temporarily diverges beyond a statistical threshold, bet on it reverting back toward its historical norm.

Why pairs trading for this specific format:

- It's a well-understood, statistically grounded technique - not a black-box LLM "vibes" call - which fits a background in statistics and gives judges something concrete to evaluate.
- It's market-neutral in principle (long one side, short the other), which reduces exposure to broad market direction over a short, unpredictable week.
- It naturally produces a testable, falsifiable signal (a z-score crossing a threshold), rather than a vague "the agent decided to buy."

Why parameters were loosened versus a "textbook" pairs strategy: A traditional pairs strategy might use a 2.0+ z-score entry and hold for weeks or months. With only 7 days, an overly conservative strategy risks generating too few signals to produce any meaningful P&L or trading history at all. The entry threshold was deliberately loosened ($z \geq 1.5$ instead of 2.0+) to generate enough trade opportunities in the window — while risk controls (stop-losses, position sizing, drawdown limits) were kept strict, so that "more trades" doesn't mean "less discipline." This distinction, loosening signal sensitivity while keeping risk management tight, is the core design tradeoff behind the whole strategy.

2. The Math, Explained Plainly

- **Correlation filter (≥ 0.80):** a first, cheap pass to narrow ~195 stocks down to pairs that move similarly, before running a more expensive test.
 - **Cointegration test (Engle-Granger, $p \leq 0.10$):** correlation alone can be misleading (two unrelated things can look correlated by chance). Cointegration checks something stronger, that the *spread* between two prices tends to revert to a stable long-run relationship, which is the actual statistical basis pairs trading relies on.
 - **Hedge ratio:** found via simple linear regression of one stock's price on the other. It answers "how many shares/units of B offset one unit of A" so the spread is measuring the *relationship*, not just raw price differences.
 - **Z-score:** measures how many standard deviations today's spread is from its own recent (60-day) average. A z-score of 1.5 means the spread is unusually stretched relative to its normal range, the trigger for a trade.
 - **Mean reversion exit:** close the position once the z-score returns to roughly 0 (the spread is back to "normal").
-

3. Risk Management ("Risk Gates")

This is a required, explicitly judged part of the submission, so it's worth stating plainly and specifically, not just in the abstract.

Control	Value	Purpose
Position sizing	2.5% of capital per trade	Limits damage from any single bad trade
Max concurrent pairs	6	Caps total portfolio exposure at any one time
Stop-loss (statistical)	Exit if z-score moves to 3.0 against the position	Cuts trades where the thesis is clearly wrong, not just "unlucky"
Stop-loss (time-based)	Exit after 3 days without reversion	Given the 7-day window, a stalled trade wastes capital and risk budget it can't afford to tie up
Soft drawdown warning	8% portfolio drawdown	Halves new position sizing, a graduated response, not all-or-nothing
Hard drawdown halt	12% portfolio drawdown	Stops all new trades entirely, a circuit breaker for when something is systematically wrong
Defined-risk structure	Options spreads (not naked positions)	Caps maximum possible loss per trade <i>by construction</i> , independent of whether the stop-loss triggers in time

Design note for the write-up: the two-tier drawdown response (soft warning at 8% → reduced sizing, hard halt at 12% → full stop) is a deliberate choice to demonstrate graduated

risk response rather than a single on/off switch, which is a stronger and more realistic risk management story than a flat percentage alone.

4. Options Implementation

The hackathon requires all strategies to incorporate options trading. Rather than treating this as a checkbox, the pairs view is *expressed* through options:

- **Expect A to outperform B** → buy a **bull call spread** on A, buy a **bear put spread** on B
- **Expect A to underperform B** → the reverse

Both are **defined-risk, multi-leg spreads** (buy one option, sell another against it), not naked long options. This means the maximum possible loss on any trade is capped by the spread's structure itself, the risk gate isn't just "hope the stop-loss order fires in time," it's baked into the position from the moment it's opened.

5. Technical Implementation

Stack: standalone Python script (not an LLM-agent-driven chat loop), calling Alpaca's official Python SDK (`alpaca-py`) directly against the Trading API and Market Data API.

Why a standalone script rather than a chat-driven agent: more testable, more auditable, and better demonstrates real engineering, each piece of logic (signal generation, risk checks, order construction) can be verified independently, which was actually done during development (see Section 6).

Pipeline (runs once per day):

1. **Universe**, a curated list of ~200 large-cap S&P 500 tickers, verified live against Alpaca's own tradable-assets API (`get_all_assets`) so the universe only includes names actually tradable right now, no dependency on external sites like Wikipedia for this step.
2. **Pair discovery**, pulls 60 days of daily price history via Alpaca's Market Data API, filters by correlation, then tests cointegration (statsmodels' Engle-Granger implementation) on the survivors.
3. **Signal evaluation**, computes the rolling z-score for each candidate pair and classifies it into an action (enter, exit, stop, hold).
4. **Risk check**, before any new trade, the risk manager confirms the portfolio isn't in a drawdown halt and hasn't hit the max concurrent position limit.
5. **Options construction**, translates the directional pairs signal into the appropriate spread structure, then queries Alpaca's live options chain to resolve real, tradable contracts closest to the target strikes/expiry.

6. **Execution**, submits the multi-leg spread order via Alpaca's Trading API.
7. **Logging**, every decision (enter/exit/stop/skip and why) is written to a structured log automatically, which is the source for this documentation and the eventual write-up.

Run cadence: entries are evaluated once daily (matches the strategy's 60-day statistical basis, no benefit to intraday polling), while the drawdown/risk check is designed to run more frequently, so a fast-moving options position can't silently blow past the drawdown limit between daily runs.

6. Known Limitations (worth stating honestly in the write-up)

Being upfront about these is a strength, not a weakness, it shows judgment about what a 7-day hackathon build can and can't achieve:

- **No formal backtest.** Given the short competition window, historical backtesting has limited predictive value for the *specific* week being traded (past cointegration doesn't guarantee it holds this week), so time was prioritized elsewhere. Parameters are informed by standard pairs- trading practice, not empirically validated against this exact universe.
 - **Static hedge ratio.** Computed once via OLS regression rather than updated dynamically, a known simplification versus more sophisticated implementations that let the hedge ratio drift over time.
 - **No explicit slippage/spread-cost modeling.** Options bid-ask spreads can be wide; the strategy's theoretical entry price may differ from the actual fill price. Not currently modeled, though position sizing provides some buffer.
 - **Static universe list.** The ~200-ticker S&P 500 candidate list is a curated snapshot rather than a live index feed, verified for tradability but not for exact current index membership.
-

7. Build Log (chronological, for reference)

- **Setup:** Enrolled solo, dedicated \$100k paper trading account created, Discord joined, team of 1 created on lablab.ai.
- **Strategy design:** Landed on pairs trading with loosened entry filters, strict risk gates, options-based expression. Parameters: 60-day lookback, $z=1.5$ entry, $z=3.0/3$ -day stop, 12% hard drawdown / 8% soft warning, 2.5% position sizing.
- **Build, Day 1 (Aug 28):** Full pipeline built and debugged live:
 - Fixed a Wikipedia 403 error by switching the universe source entirely to a static list verified against Alpaca's own tradable-assets API.
 - Fixed a market-data feed error (free/paper accounts only have access to the IEX feed, not the paid SIP feed).

- Fixed options contract resolution to search a date *window* around the target expiry rather than one exact date, since options only expire on specific real dates.
 - Added graceful error handling so a single problematic pair (e.g. no listed options, or an order rejected outside market hours) is skipped and logged rather than crashing the whole run.
 - First successful end-to-end run: discovered 1,207 correlated candidates from 195 verified tickers, narrowed to 25 cointegrated pairs, and generated real entry signals (PCAR/TRV, MDT/MSI), with MDT/MSI successfully resolving to real, tradable option contracts.
-

8. Bugs Found and Fixed (Day 1)

Documenting these honestly, catching and fixing real issues during development is normal engineering, and it's better material for a write-up than pretending everything worked on the first try.

Issue	Symptom	Fix
Wikipedia scraping blocked	<code>HTTPError: 403 Forbidden</code> when fetching S&P 500 tickers	Replaced live scraping entirely with a static curated ticker list, verified against Alpaca's own <code>get_all_assets()</code> API for current tradability, removes the external dependency altogether.
Wrong market data feed	<code>subscription does not permit querying recent SIP data</code>	Paper/free accounts only have access to the IEX data feed, not the paid SIP feed. Fixed by explicitly requesting <code>DataFeed.IEX</code> in the historical bars request.
Options contract resolution returning empty	<code>ValueError: min() iterable argument is empty</code>	Options only expire on specific real dates (mostly Fridays), the code was requesting one exact computed date that often isn't a real expiry. Fixed by querying a date <i>window</i> (± 5 days) and picking the closest actual available expiry, then the closest strike within it.
Single pair failure crashing the whole run	One bad pair (e.g. no listed options) would stop the entire daily cycle	Wrapped options resolution and order placement in error handling that logs a warning and skips just that pair, letting the rest of the run continue normally.

Orders rejected outside market hours	options market orders are only allowed during market hours	Not a bug, but handled the same way, caught, logged, and skipped gracefully rather than crashing, since options (like stocks) only trade 9:30 AM-4:00 PM ET.
No memory between runs (most significant fix)	Every run created a brand-new, empty risk tracker — meaning it would forget any position opened in a previous run, risking duplicate entries on the same signal and making stop-loss/take-profit checks impossible (a "forgotten" position can never be checked for exit conditions)	Added <code>PortfolioState.save()</code> / <code>.load()</code> , writing portfolio state (open positions, equity, drawdown history) to <code>portfolio_state.json</code> after every run and reloading it at the start of the next. Now <code>main.py</code> , whether triggered manually or by a scheduled task, always knows what's currently open before deciding whether to enter, exit, or hold.
No profit-taking rule	The only exit was full statistical mean-reversion (z-score back to 0); no way to lock in a partial gain if the position had already moved favorably	Added a take-profit check (35% of premium risked) to the frequent risk-monitoring loop, which now also stores each position's actual resolved option contract symbols and entry premium so it can be evaluated against live quotes between daily runs.

Why the "no memory between runs" fix mattered most: the whole point of having a fast risk-check loop (running every 30 minutes via Windows Task Scheduler, separate from the once-daily entry evaluation) is to catch stop-losses and take-profits promptly. Without a persistent state, that loop had nothing to check, it would start from zero every single time. This was found and fixed before any real trade was placed, so it never actually caused a duplicate entry or a missed stop in practice, but it would have, had it gone live before the fix.

9. Automation

The agent runs on a schedule via Windows Task Scheduler rather than being manually triggered each day:

- **Daily entry check**, runs `main.py` once a day during market hours, evaluating all pairs for new entry signals and checking existing positions for the z-score/time-based stop and mean-reversion exit.
- **Risk check (every 30 min during market hours)**, runs `main.py --risk-check`, monitoring portfolio drawdown and checking each open position against the take-profit target, independent of the daily cycle.

The day number used in logs and decisions is calculated automatically from the competition start date, so the scheduled tasks never need manual editing of a date or day-count argument.

10. Live Trading Results & Further Fixes (Day 3-4 onward)

First real fills: Monday, Sep 1 (competition Day 4) was the first trading day the universe expansion and widened options-expiry window (see prior section) actually took effect on. The agent successfully opened 5 real option spread positions simultaneously, a meaningful jump from Day 1's zero fills, confirming the expanded universe (348 tickers) and wider expiry search window (± 12 days) resolved the earlier "no option contracts found" false negatives on otherwise perfectly optionable large-cap names.

Pairs opened Day 4: CF/CSX (bear put spread on CF + bull call spread on CSX, $z=2.02$), plus 4 additional pairs opened the same session.

Bugs found and fixed from live trading

Live trading surfaced three further real bugs that no amount of dry-run testing had caught, because they only manifest once real broker fills and real position closes are involved:

Issue	Root cause	Fix
Take-profit close failed with a 404	<code>close_position()</code> was closing by <i>underlying ticker</i> (e.g. "CF"), but options positions live under their own contract symbol (e.g. <code>CF260918P00129000</code>), the underlying stock was never actually held, so this call could never succeed for an options-based position.	Added <code>close_spread_legs()</code> , which closes a spread with one reverse multi-leg order addressed by the actual option contract symbols, and skips any leg the broker doesn't hold rather than erroring the whole close out.
A position existed in local state but not at the broker	Orders were recorded as "open" in <code>portfolio_state.json</code> immediately after submission, without confirming every leg actually filled. One leg of the CF spread apparently didn't fill as expected, leaving a state/broker mismatch.	Added <code>verify_legs_filled()</code> , which polls for real fills before a new entry is written to state; unfilled legs are unwound and the entry is skipped rather than recorded as a phantom position.

Strike collision on wide strike-increment underlyings (e.g. GD)	<code>options_builder.py</code> picks target strikes as a % of price, then <code>resolve_option_contracts()</code> snaps each leg independently to the nearest real listed strike, for underlyings with wide strike increments, both legs could snap to the <i>same</i> real strike, which the broker correctly rejected ("all legs must have unique symbols").	<code>resolve_option_contracts()</code> now tracks strikes already claimed within a spread and falls back to the next-closest distinct strike instead of colliding.
--	---	---

Also added: `reconcile_with_broker()` in `risk_manager.py`, run at the start of each check, automatically drops any state entry with zero legs left at the broker, and logs (without guessing at a cause) any partial mismatch it finds. This makes the system self-correcting for this class of issue going forward, rather than only patching the one incident.

First realized trade result: the CF/CSX pair partially closed on a take-profit trigger within roughly 31 minutes of opening, realizing a loss of approximately \$100 on that leg. Given the position was opened with a genuine statistical signal ($z=2.02$) and the very short hold time before the close, this was investigated rather than dismissed as random noise, the investigation is what surfaced the three bugs above. Worth noting honestly in the write-up: the fast, unexpected close was a *symptom* of the fill-verification and close-targeting bugs, not a strategy failure.

Status as of Sep 1 (Day 4)

- 5 positions opened Monday Aug 31, still open (within the 3-day max hold window, expected to resolve by ~Sep 3)
- Portfolio drawdown tracking in the 2.5-3.3% range, comfortably within the 8% soft warning and 12% hard halt thresholds; normal variation expected right after deploying most of the per-trade risk budget across 5 simultaneous entries, not a sign of a problem
- **Still open / not yet done:** structured P&L logging on each close (a clean "trades closed, win/loss, net \$" summary, currently only visible by reading raw fill data on Alpaca's dashboard); a review of `spread_width_pct` given the strike-collision issue surfaced on wider-increment underlyings

Concentration risk fix: ticker overlap across concurrent pairs

Reviewing the 5 open positions revealed that **CBRE appeared as one leg in three separate concurrent pairs** (CBRE/ROP, CBRE/IQV, CBRE/LH), the risk manager sized each pair independently without checking whether an underlying was already committed elsewhere, so actual exposure to CBRE was effectively 3x the intended 2.5% per-trade size.

Fix: `PortfolioState.tickers_in_use()` now tracks every underlying across all open pairs, and `can_open_new_position()` blocks any new entry that would reuse a ticker already committed to a different open position. Tested against the exact CBRE scenario

before deployment, a new pair sharing a ticker with an open position is correctly rejected with a clear reason logged, while non-overlapping pairs are unaffected. Existing positions opened before this fix were left as-is (only new entries are gated going forward).